

IF25-22017

PENGEMBANGAN APLIKASI MOBILE

PERTEMUAN 11

Project Sprint 1: Planning dan Setup

Requirements, Architecture, Repository, dan CI/CD

Program Studi Teknik Informatika
Institut Teknologi Sumatera
Tahun Akademik Genap 2025/2026



SELAMAT DATANG DI FASE PROJECT!

Mulai minggu ini hingga UAS, kalian akan mengerjakan Project Akhir secara berkelompok (2-3 orang). Project ini mengaplikasikan semua materi dari Pertemuan 1-10.

Timeline Project:

W11	Sprint 1	Planning dan Setup
W12	Sprint 2	Core Features
W13	Sprint 3	Advanced Features
W14	Sprint 4	Polish & Testing
W15	Sprint 5	Final Preparation
W16	UAS	Final Demo Day

OUTLINE PERTEMUAN

01

Review & Recap

Rangkuman materi Pertemuan 1-10

02

Project Requirements

Scope, features, dan kriteria penilaian

03

Architecture Design

Clean Architecture dan module structure

04

Repository Setup

GitHub, branching, dan collaboration

05

CI/CD Basic

GitHub Actions untuk project

06

Sprint Planning

Task breakdown dan timeline

CAPAIAN PEMBELAJARAN

CPMK0501

Menerapkan konsep pemrograman untuk pengembangan aplikasi mobile multiplatform

CPMK0502

Menggunakan berbagai framework dan library dalam pengembangan aplikasi

CPMK0503

Menerapkan prinsip UI/UX dalam pengembangan aplikasi mobile

Setelah pertemuan ini, mahasiswa mampu:

- ✓ Merencanakan project dengan requirements yang jelas
- ✓ Mendesain arsitektur aplikasi yang scalable
- ✓ Setup repository dan CI/CD pipeline

BAGIAN 1

REVIEW DAN RECAP

Rangkuman Materi Pertemuan 1-10

RECAP: FOUNDATION (Week 1-5)

W1

Setup Environment

Android Studio, KMP Project Structure, Gradle

W2

Kotlin Advanced

Coroutines, Flow, Extension Functions, Sealed Classes

W3

Compose Basics

Composables, Modifiers, Layouts, Material 3

W4

State & MVVM

remember, StateFlow, ViewModel, UDF Pattern

W5

Navigation

NavHost, NavController, Arguments, Bottom Nav

RECAP: ADVANCED (Week 6-10)

W6

Networking

Ktor Client, REST API, JSON Serialization, Repository

W7

Local Storage

SQLDelight, DataStore, Offline-First, Sync

W8

Platform Code

expect/actual, Platform APIs, Permissions

W9

AI Integration

OpenAI API, Gemini API, Prompt Engineering

W10

Testing & DI

Koin, kotlin.test, MockK, Turbine, Compose Test

TECH STACK UNTUK PROJECT

Framework	Kotlin Multiplatform, Compose Multiplatform
Architecture	MVVM, Clean Architecture, Repository Pattern
Async	Coroutines, Flow, StateFlow
Networking	Ktor Client, Kotlinx Serialization
Storage	SQLDelight, DataStore Preferences
DI	Koin
Testing	kotlin.test, MockK, Turbine, Compose Test

BAGIAN 2

PROJECT REQUIREMENTS

Scope, Features, dan Kriteria

PROJECT OVERVIEW



Buat aplikasi mobile multiplatform yang menyelesaikan masalah nyata

Aplikasi harus berjalan di Android (wajib) dan iOS (opsional). Gunakan semua teknologi yang sudah dipelajari: KMP, Compose, MVVM, Networking, Local Storage, dan Testing.

Ketentuan:

- Tim: 2-3 orang per kelompok
- Platform: Android (wajib), iOS (bonus)
- Durasi: 5 minggu (Sprint 1-5)
- Presentasi: UAS Demo Day (Minggu 16)
- Bobot: 60% dari nilai akhir (5 Sprint x 5% + UAS 35%)



Plagiarisme = nilai 0 untuk seluruh tim. Kode harus original atau dengan proper attribution.

MINIMUM REQUIREMENTS

UI/UX

Minimal 5 screens, responsive layout, Material Design 3

Architecture

MVVM pattern, Clean Architecture layers

Data

REST API integration ATAU local database dengan CRUD

State

Proper state management dengan StateFlow

Navigation

Multi-screen navigation dengan argument passing

Testing

Minimal 10 unit tests, 3 UI tests, coverage > 50%

DI

Koin dependency injection setup

BONUS FEATURES (+Nilai)

iOS Support	+10%	App berjalan di iOS simulator/device
AI Integration	+10%	Integrasi OpenAI/Gemini API
Offline First	+5%	App berfungsi tanpa internet
Dark Mode	+5%	Support light/dark theme
Animations	+5%	Meaningful animations dan transitions
CI/CD	+5%	Automated build dan test dengan GitHub Actions
Play Store Ready	+5%	Signed APK/AAB siap upload



Maksimal bonus: +45%. Pilih bonus yang sesuai dengan kemampuan dan waktu tim.

IDE PROJECT

Task Manager

Todo app dengan categories, priorities, reminders



Expense Tracker

Track pengeluaran, budgeting, statistics



Recipe App

Browse, save, dan share resep masakan



News Reader

Aggregator berita dengan categories dan bookmarks



Fitness Tracker

Log workout, track progress, set goals



Study Buddy

Flashcards, quiz, study timer dengan AI



Local Events

Discover events nearby, RSVP, notifications



Boleh ide sendiri! Konsultasikan dengan dosen untuk approval.

BAGIAN 3

ARCHITECTURE DESIGN

Clean Architecture dan Module Structure

CLEAN ARCHITECTURE

PRESENTATION LAYER

UI (Composables), ViewModels, UI State



DOMAIN LAYER

Use Cases, Domain Models, Repository Interfaces



DATA LAYER

Repository Impl, Data Sources (Remote/Local), DTOs

Dependency Rule: Dependencies point inward. Domain tidak tahu tentang Data atau Presentation.

Benefits: Testable • Maintainable • Scalable • Separation of Concerns

RECOMMENDED PROJECT STRUCTURE

```
composeApp/  
└─ src/  
    └─ commonMain/kotlin/com/example/app/  
        └─ App.kt # Entry point  
        └─ di/ # Koin modules  
            └─ AppModule.kt  
            └─ DataModule.kt  
            └─ ViewModelModule.kt  
        └─ data/ # Data layer  
            └─ local/ # SQLDelight, DataStore  
            └─ remote/ # Ktor, API services  
            └─ repository/ # Repository implementations  
            └─ model/ # DTOs, Entities  
        └─ domain/ # Domain layer  
            └─ model/ # Domain models  
            └─ repository/ # Repository interfaces  
            └─ usecase/ # Use cases (optional)  
        └─ presentation/ # Presentation layer  
            └─ navigation/ # NavHost, Routes  
            └─ theme/ # Colors, Typography  
            └─ components/ # Reusable composables  
            └─ screens/ # Feature screens  
                └─ home/  
                └─ detail/  
                └─ settings/
```

MVVM IMPLEMENTATION

ViewModel

```
class HomeViewModel(
    private val repository: NoteRepository
) : ViewModel() {

    private val _uiState = MutableStateFlow<HomeUiState>(
        HomeUiState.Loading
    )
    val uiState: StateFlow<HomeUiState> = _uiState

    init { loadNotes() }

    private fun loadNotes() {
        viewModelScope.launch {
            repository.getAllNotes()
                .collect { notes ->
                    _uiState.value = HomeUiState.Success(notes)
                }
        }
    }
}
```

UI State

```
sealed interface HomeUiState {
    data object Loading : HomeUiState
    data class Success(
        val notes: List<Note>
    ) : HomeUiState
    data class Error(
        val message: String
    ) : HomeUiState
}

// In Composable
@Composable
fun HomeScreen(vm: HomeViewModel = koinViewModel()) {
    val uiState by vm.uiState.collectAsState()

    when (val state = uiState) {
        is HomeUiState.Loading -> LoadingIndicator()
        is HomeUiState.Success -> NotesList(state.notes)
        is HomeUiState.Error -> ErrorMessage(state.message)
    }
}
```

💡 Best Practices:

- Sealed class untuk UI state - exhaustive when
- StateFlow untuk state, SharedFlow untuk one-time events
- ViewModel tidak tahu tentang UI/Compose
- Collect state dengan collectAsState() di Composable

BAGIAN 4

REPOSITORY SETUP

GitHub, Branching, dan Collaboration

GITHUB REPOSITORY SETUP

1

Create Repository

github.com/new → Private repo → Add README

2

Add Collaborators

Settings > Collaborators > Add team members

3

Branch Protection

Settings > Branches > Add rule for 'main'

4

Clone & Push

git clone → copy project → git push

 Naming: [team-name]-[project-name] contoh: team-alpha-expense-tracker

BRANCHING STRATEGY

main

Production-ready code. Protected branch, requires PR review.

↑ merge

develop

Integration branch. Features merged here first.

↑ merge

feature/*

Individual feature branches: feature/home-screen, feature/api-integration

```
# Workflow
git checkout develop
git pull origin develop
git checkout -b feature/home-screen
# ... work on feature ...
git add . && git commit -m "feat: implement home screen"
git push origin feature/home-screen
# Create Pull Request on GitHub → Request review → Merge
```

COMMIT CONVENTIONS

Format: <type>: <description>

feat	New feature	feat: add note creation screen
fix	Bug fix	fix: resolve crash on empty list
refactor	Code refactoring	refactor: extract common composables
test	Adding tests	test: add unit tests for repository
docs	Documentation	docs: update README with setup guide
style	Code style/formatting	style: apply ktlint formatting
chore	Maintenance tasks	chore: update dependencies



Good commits = easier code review + better git history

PULL REQUEST WORKFLOW



Create PR

Push branch → Open PR → Fill template



Request Review

Assign reviewer (teammate)



Address Feedback

Fix issues, push updates



Approve & Merge

Get approval → Squash & merge

PR Template

```
## Description
Briefly describe what this PR does.

## Changes
- [ ] Added home screen UI
- [ ] Implemented ViewModel
- [ ] Added unit tests

## Screenshots (if UI changes)
[Attach screenshots here]

## Checklist
- [ ] Code compiles without errors
- [ ] Tests pass
- [ ] No warnings
```

BAGIAN 5

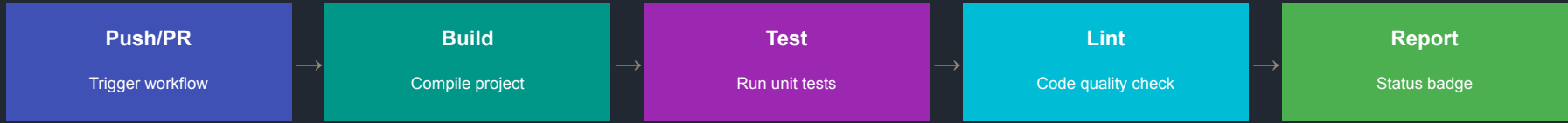
CI/CD BASIC

GitHub Actions untuk Project

CI/CD UNTUK PROJECT

CI/CD memastikan setiap push/PR di-build dan di-test secara otomatis. Catch bugs early, maintain code quality.

Workflow untuk Project:



.github/workflows/ci.yml

```
name: CI
on: [push, pull_request]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { java-version: '17', distribution: 'temurin', cache: gradle }
      - run: chmod +x gradlew
      - run: ./gradlew test
      - run: ./gradlew assembleDebug
```

STATUS BADGE & BRANCH PROTECTION

Add Status Badge ke README:

```
# Project Name  
  
![CI](https://github.com/[owner]/[repo]/actions/workflows/ci.yml/badge.svg)
```

Setup Branch Protection:

Settings > Branches > Add branch protection rule

Branch name pattern: main (atau develop)

- ✓ Require a pull request before merging
- ✓ Require status checks to pass (select CI workflow)
- ✓ Require conversation resolution before merging

✓ Dengan branch protection, tidak ada yang bisa push langsung ke main tanpa PR dan passing tests!

BAGIAN 6

SPRINT PLANNING

Task Breakdown dan Timeline

SPRINT OVERVIEW

Sprint 1 (W11)

Planning & Setup

Requirements, architecture, repo, CI/CD

Sprint 2 (W12)

Core Features

Main screens, navigation, data layer

Sprint 3 (W13)

Advanced Features

API integration, additional features

Sprint 4 (W14)

Polish & Testing

UI polish, bug fixes, testing

Sprint 5 (W15)

Final Prep

Final fixes, demo preparation

SPRINT 1: PLANNING & SETUP (HARI INI)

☐	Form kelompok	2-3 orang, tentukan role (lead, dev, QA)	15 min
☐	Pilih project idea	Brainstorm, pilih, approval dosen	20 min
☐	Define requirements	List fitur minimum dan bonus	20 min
☐	Design architecture	Gambar diagram, tentukan struktur folder	15 min
☐	Create repository	GitHub repo, add collaborators	10 min
☐	Setup project	Clone, create KMP project, push	20 min
☐	Setup CI/CD	GitHub Actions workflow	15 min
☐	Create project doc	README dengan requirements dan timeline	15 min

PROJECT DOCUMENTATION

README.md

```
# Project Name
![[CI]](badge-url)

## Team
- Member 1 (Lead) - @github
- Member 2 (Dev) - @github
- Member 3 (QA) - @github

## Description
Brief description of the app and its purpose.

## Features
### Minimum
- [ ] Feature 1
- [ ] Feature 2

### Bonus
- [ ] Bonus feature 1

## Tech Stack
KMP, Compose, Ktor, SQLDelight, Koin

## Architecture
[Architecture diagram/description]

## Setup
1. Clone repo
2. Open in Android Studio
3. Run on emulator/device
```



HANDS-ON: SPRINT 1 EXECUTION

Waktu: Sisa pertemuan hari ini (~60 menit)

Step-by-Step:

1	10 min	Form kelompok dan pilih project idea
2	15 min	Diskusi requirements - list fitur minimum dan bonus
3	5 min	Create GitHub repository, add collaborators
4	15 min	Setup KMP project, push initial code
5	10 min	Setup GitHub Actions CI workflow
6	5 min	Write README dengan project info

✓ Target: Setiap kelompok punya repo dengan CI passing sebelum pulang!

DELIVERABLES SPRINT 1

Bobot: 5% | Deadline: Sebelum Pertemuan 12



Deliverables:

- ☐ GitHub repository dengan semua team members sebagai collaborator
- ☐ KMP project structure dengan proper architecture folders
- ☐ Working GitHub Actions CI (build + test passing)
- ☐ Comprehensive README.md dengan: team info, project description, features list, tech stack, architecture overview
- ☐ Project plan document (bisa di README atau separate): fitur per sprint, task assignment



Submission:

- Submit GitHub repo URL ke e-learning
- Pastikan dosen di-add sebagai collaborator (untuk review)
- CI badge harus showing 'passing'

RUBRIK PENILAIAN SPRINT 1

Komponen	Bobot	Kriteria
Repository Setup	20%	Proper naming, collaborators added
Project Structure	25%	Clean architecture folders, builds
CI/CD Pipeline	20%	GitHub Actions working, badge shows
Documentation	25%	Complete README, project plan
Team Collaboration	10%	All members contributed (commits)
★ Bonus: Koin DI setup (+10%)		⚠ Late: -10%/day

PREVIEW: SPRINT 2 (MINGGU DEPAN)

Sprint 2: Core Features Implementation

- **Implement main screens**

Home, Detail, List screens dengan Compose

- **Setup navigation**

NavHost, routes, argument passing

- **Create data layer**

Repository, data sources, models

- **Basic CRUD operations**

Create, Read, Update, Delete functionality

🚀 **Persiapan: Selesaikan Sprint 1, review materi Week 3-5 (Compose, State, Navigation)**

TIPS FOR PROJECT SUCCESS



Start Early

Jangan menunda, 5 minggu berlalu cepat



Communicate

Daily standup dengan tim, gunakan Discord/WhatsApp



Commit Often

Small, frequent commits lebih baik dari big bang



Document

Tulis README, comment code, buat diagram



Test Early

Write tests as you code, bukan di akhir



Ask for Help

Konsultasi dosen jika stuck, jangan tunggu deadline

COMMON MISTAKES TO AVOID

X Scope Creep

→ Stick to requirements, add bonus only if time permits

X Poor Git Hygiene

→ Use branches, write good commits, review PRs

X No Architecture

→ Plan structure first, don't just code randomly

X Ignoring Tests

→ Write tests alongside features, not after

X Last Minute Rush

→ Consistent progress each sprint, track in GitHub Projects

X One Person Army

→ Distribute work, everyone should contribute code

RESOURCES

Docs

KMP Documentation

kotlinlang.org/docs/multiplatform.html

Docs

Compose Multiplatform

jetbrains.com/lp/compose-multiplatform

Guide

GitHub Actions

docs.github.com/en/actions

Tool

Figma (UI Design)

figma.com

Tool

Excalidraw (Diagrams)

excalidraw.com



TEAM FORMATION TIME!

Form kelompok sekarang (2-3 orang)

Informasi yang perlu dikumpulkan:

- Nama kelompok (creative!)
- Anggota 1: Nama, NIM, GitHub username, Role
- Anggota 2: Nama, NIM, GitHub username, Role
- Anggota 3: Nama, NIM, GitHub username, Role (opsional)
- Ide project (top 2-3 choices)
- Contact person (WhatsApp/Discord)



Waktu: 10 menit

Setelah selesai, laporkan ke dosen untuk mendapat persetujuan project idea.



SESI TANYA JAWAB

Ada pertanyaan tentang Project?

CHECKLIST SEBELUM PULANG

- ☐ Tim sudah terbentuk (2-3 orang)
- ☐ Project idea sudah disetujui dosen
- ☐ GitHub repository sudah dibuat
- ☐ Semua anggota sudah di-add sebagai collaborator
- ☐ Initial KMP project sudah di-push
- ☐ GitHub Actions CI sudah setup dan passing
- ☐ README.md sudah ada dengan info dasar
- ☐ Paham deadline dan deliverables Sprint 1

Let's Build Something Amazing!

Sprint 1: Planning dan Setup

*"The secret of getting ahead
is getting started."*

- Mark Twain

Good luck with your projects! 